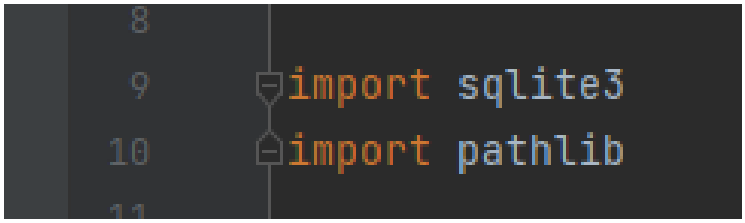
Criterion C: Development

List of techniques used:

- Additional libraries
- Database creation
- Table creation
- Inputted data into a database
- Data extraction from the database
- Mean calculations
- Use of 2D arrays
- Continue statement
- Error checking
- For loops

Use of additional libraries:







The “sqlite3” library was used to easily create and manage databases.

“Pathlib” was used to detect if the file already exists and if it was the first run.

“Tabulate” was used to create nicely formatted tables.

“Numpy” was used to easily make calculations such as the mean.

Database creation:

```python
FILENAME = "stats.db"

# TEST if the FILENAME already exists

FIRST_RUN = True

if (pathlib.Path.cwd() / FILENAME).exists():
    FIRST_RUN = False

CONNECTION = sqlite3.connect(FILENAME)
CURSOR = CONNECTION.cursor()
```

One database was created to store multiple tables for players and a singular team table. On the first run a database is created titled “stats.db.” The pathlib function checks if the file already exists.

Table creation:

```python
def setupTable(NAME):
    """
    Create a database table for a new player
    :param NAME: str
    :return:
    """
    global CURSOR, CONNECTION

    CURSOR.execute(f"""
        CREATE TABLE
            {NAME} (
                id INTEGER PRIMARY KEY,
                date TEXT NOT NULL,
                opponent TEXT NOT NULL,
                minutes INTEGER,
                points INTEGER,
                rebounds INTEGER,
                assists INTEGER,
                steals INTEGER,
                blocks INTEGER,
                turnovers INTEGER,
                fg_made INTEGER,
                fg_attempted INTEGER,
                fg_per FLOAT,
                three_point_made INTEGER,
                three_point_attempted INTEGER,
                three_point_per FLOAT,
                ft_made INTEGER,
                ft_attempted INTEGER,
                ft_per FLOAT,
                fouls INTEGER
            )
    ;""")

    CONNECTION.commit()

    print(f"{NAME} added to database!")
```

```python
def setupTeamTable():
    """
    Set up a table for the team stats
    :return:
    """

    global CURSOR, CONNECTION

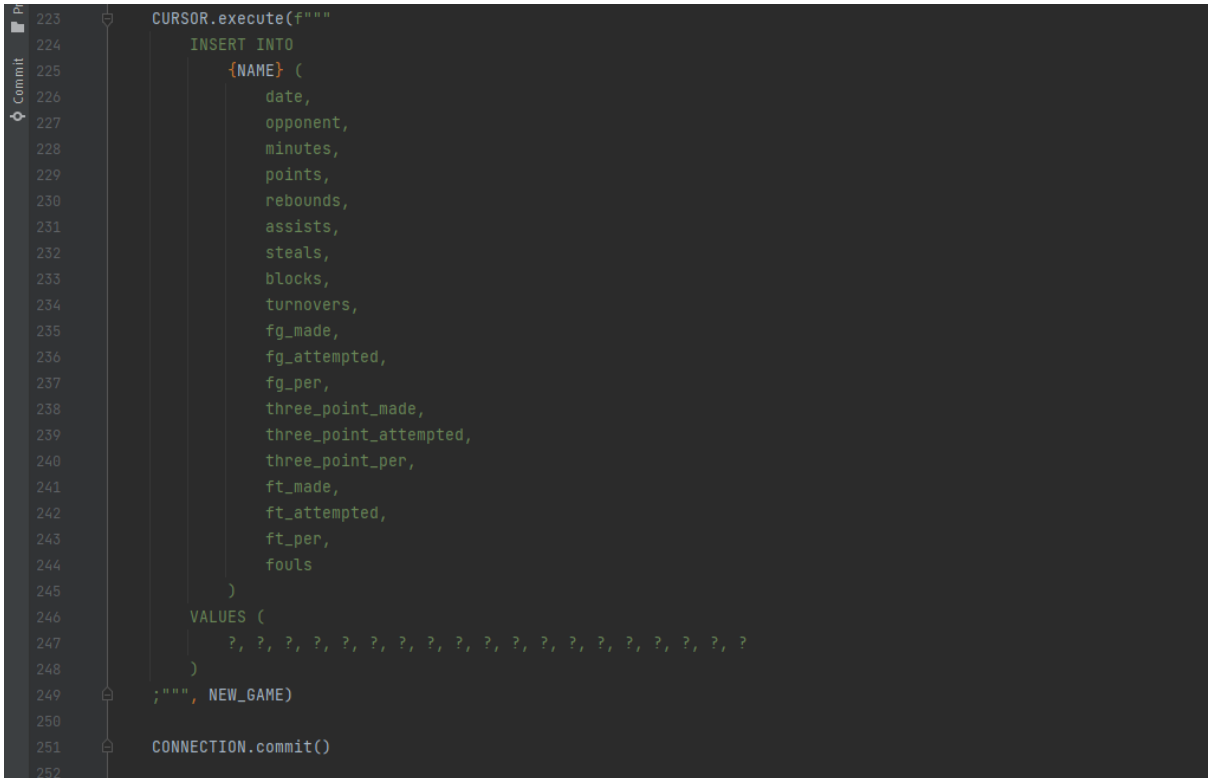
    CURSOR.execute("""
        CREATE TABLE
            Lillian_Osborne_Basketball (
                id INTEGER PRIMARY KEY,
                name TEXT NOT NULL,
                avg_minutes INTEGER,
                avg_points INTEGER,
                avg_rebounds INTEGER,
                avg_assists INTEGER,
                avg_steals INTEGER,
                avg_blocks INTEGER,
                avg_turnovers INTEGER,
                avg_fg_made INTEGER,
                avg_fg_attempted INTEGER,
                avg_fg_per FLOAT,
                avg_three_point_made INTEGER,
                avg_three_point_attempted INTEGER,
                avg_three_point_per FLOAT,
                avg_ft_made INTEGER,
                avg_ft_attempted INTEGER,
                avg_ft_per FLOAT,
                avg_fouls INTEGER
                )
    ;""")

    CONNECTION.commit()
```

Using the inputted player name, the program creates a table within the database using the player name as the name of the table. The program also creates a singular team table on the first run. The name, date, and opponent values cannot be null while the other rows can be left as blank or 0.

Inputted data into a database:

```python
def getPlayerName():
    """
    User inputs the name of the player
    :return: str
    """
    FIRST_NAME = input("Enter first name: ")
    LAST_NAME = input("Enter last name: ")
    PLAYER = FIRST_NAME + "_" + LAST_NAME
    PLAYERS.append(PLAYER)
    return PLAYER
```



```
CURSOR.execute(f"""
    INSERT INTO
        {NAME} (
            date,
            opponent,
            minutes,
            points,
            rebounds,
            assists,
            steals,
            blocks,
            turnovers,
            fg_made,
            fg_attempted,
            fg_per,
            three_point_made,
            three_point_attempted,
            three_point_per,
            ft_made,
            ft_attempted,
            ft_per,
            fouls
        )
    VALUES (
        ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
    )
;""", NEW_GAME)

CONNECTION.commit()
```

The user inputs information for multiple functions of the program. The user inputs the name of the player as well as the values for the player stats. The user enters the name of the player when they choose to add a new player. The first and last name is joined using an underscore to be used as a singular string. When the user chooses to edit the player, they can choose to add a new row, where they are prompted to enter a value for every column. The program has some error checking to filter out bad values entered. The percentage is calculated by dividing the made shots by the attempts.

Data extraction from the database:

```python
def displayPlayer(NAME):
    """
    Display the table for a player
    :param NAME: str
    :return:
    """

    global CURSOR, CONNECTION

    from tabulate import tabulate

    STATS = CURSOR.execute(f"""
            SELECT
                *
            FROM
                {NAME}
        """).fetchall()

    TABLE = STATS
    HEADER = ['DATE', 'OPP', 'MIN', 'PTS', 'REB', 'AST', 'STL', 'BLK', 'TO', 'FGM', 'FGA', 'FG%', '3PM', '3PA', '3P%',
              'FTM', 'FT_A', 'FT%', 'F']

    print(tabulate(TABLE, headers=HEADER))
```

```python
def displayTeam():
    """
    Display the table for a player
    :param NAME: str
    :return:
    """

    global CURSOR, CONNECTION

    from tabulate import tabulate

    STATS = CURSOR.execute(f"""
            SELECT
                *
            FROM
                Lillian_Osborne_Basketball
        """).fetchall()

    TABLE = STATS
    HEADER = ['NAME', 'MPG', 'PPG', 'RPG', 'APG', 'SPG', 'BPG', 'TPG', 'FGM', 'FGA', 'FG%', '3PM', '3PA', '3P%',
              'FTM', 'FT_A', 'FT%', 'FPG']

    print(tabulate(TABLE, headers=HEADER))
```

The program extracts the data from each of the tables to be viewed in a formatted table by the viewer. The select * is used to select everything from the table. .fetchall() ensures that every value is selected. Tabulate is used to format all of these values into a table that is printed out and labeled headers are used for every column.

Mean calculations:

```
    import numpy as np

    STAT = np.array(STATS)

    MIN_AVG = STAT[1:, 0].mean()
    PTS_AVG = STAT[1:, 1].mean()
    REB_AVG = STAT[1:, 2].mean()
    AST_AVG = STAT[1:, 3].mean()
    STL_AVG = STAT[1:, 4].mean()
    BLK_AVG = STAT[1:, 5].mean()
    TO_AVG = STAT[1:, 6].mean()
    FGM_AVG = STAT[1:, 7].mean()
    FGA_AVG = STAT[1:, 8].mean()
    FGP_AVG = STAT[1:, 9].mean()
    TPM_AVG = STAT[1:, 10].mean()
    TPA_AVG = STAT[1:, 11].mean()
    TPP_AVG = STAT[1:, 12].mean()
    FTM_AVG = STAT[1:, 13].mean()
    FTA_AVG = STAT[1:, 14].mean()
    FTP_AVG = STAT[1:, 15].mean()
    F_AVG = STAT[1:, 16].mean()

    AVG = [MIN_AVG, PTS_AVG, REB_AVG, AST_AVG, STL_AVG, BLK_AVG, TO_AVG, FGM_AVG, FGA_AVG, FGP_AVG, TPM_AVG, TPA_AVG,
           TPP_AVG, FTM_AVG, FTA_AVG, FTP_AVG, F_AVG]

    return AVG
```

The numpy library is used to calculate the mean of each value. A variable is used for the mean of each column in one table in the database, then sent to the team table where the player's stat average is displayed.

Use of 2D arrays:

When the data is being extracted a 2D array is being used. During the table formatting the data is extracted as a 2D array and the data is displayed as a table. The data is also extracted as a 2D array to calculate the mean of the values.

Continue statement:



```
        elif CHOICE == 4:
            print("""
1. Add new game
2. Delete row
3. Delete player
4. Exit
            """)
            SUBMENU = int(input("> "))
            if SUBMENU == 1:
                NAME = getPlayerID()
                addNewRow(NAME)
                AVG = getPlayerAverage(NAME)
                updateAverage(NAME, AVG)
                updateTeamAverage(NAME, AVG)
            elif SUBMENU == 2:
                NAME = getPlayerID()
                ROW = getRowID(NAME)
                deleteRow(NAME, ROW)
            elif SUBMENU == 3:
                NAME = getPlayerID()
                deleteTable(NAME)
            else:
                continue
        else:
            exit()
```

The program uses the continue function to go back to the main menu. If the user selects the submenu, then exits the submenu, the continue function causes the program to exit the loop and return to the main menu loop.

Error checking:

```
    DATE = input("Date of game(MM/DD)> ")
    OPPONENT = input("Opponent(XXX)> ")
    MINUTES = input("Minutes played> ")
    if MINUTES == "":
        MINUTES = 0
    elif MINUTES.isnumeric() is False:
        MINUTES = input("Please enter a number> ")
    int(MINUTES)
    POINTS = input("Points> ")
    if POINTS == "":
        POINTS = 0
    elif POINTS.isnumeric() is False:
        POINTS = input("Please enter a number> ")
    int(POINTS)
    REBOUNDS = input("Rebounds> ")
    if REBOUNDS == "":
        REBOUNDS = 0
    elif REBOUNDS.isnumeric() is False:
        REBOUNDS = input("Please enter a number> ")
    int(REBOUNDS)
    ASSISTS = input("Assists> ")
    if ASSISTS == "":
        ASSISTS = 0
    elif ASSISTS.isnumeric() is False:
        ASSISTS = input("Please enter a number> ")
    int(ASSISTS)
    STEALS = input("Steals> ")
    if STEALS == "":
        STEALS = 0
    elif STEALS.isnumeric() is False:
        STEALS = input("Please enter a number> ")
    int(STEALS)
    BLOCKS = input("Blocks> ")
    if BLOCKS == "":
        BLOCKS = 0
    elif BLOCKS.isnumeric() is False:
        BLOCKS = input("Please enter a number> ")
    int(BLOCKS)
    TURNOVERS = input("Turnovers> ")
```

```
    if TURNOVERS == "":
        TURNOVERS = 0
    elif TURNOVERS.isnumeric() is False:
        TURNOVERS = input("Please enter a number> ")
    int(TURNOVERS)
    FG_MADE = input("Field goals made> ")
    if FG_MADE == "":
        FG_MADE = 0
    elif FG_MADE.isnumeric() is False:
        FG_MADE = input("Please enter a number> ")
    FG_MADE = int(FG_MADE)
    FG_ATTEMPTED = input("Field goals attempted> ")
    if FG_ATTEMPTED == 0 or "":
        FG_ATTEMPTED = 0
    elif FG_ATTEMPTED.isnumeric() is False:
        FG_ATTEMPTED = input("Please enter a number> ")
    FG_ATTEMPTED = int(FG_ATTEMPTED)
    if FG_ATTEMPTED > 0:
        FG_PER = (FG_MADE / FG_ATTEMPTED) * 100
        round(FG_PER, 1)
    elif FG_ATTEMPTED == 0:
        FG_PER = 0
    THREE_MADE = input("Threes made> ")
    if THREE_MADE == "":
        THREE_MADE = 0
    elif THREE_MADE.isnumeric() is False:
        THREE_MADE = input("Please enter a number> ")
    THREE_MADE = int(THREE_MADE)
    THREE_ATTEMPTED = input("Threes attempted> ")
    if THREE_ATTEMPTED == 0 or "":
        THREE_ATTEMPTED = 0
    elif THREE_ATTEMPTED.isnumeric() is False:
        THREE_ATTEMPTED = input("Please enter a number> ")
    THREE_ATTEMPTED = int(THREE_ATTEMPTED)
    if THREE_ATTEMPTED > 0:
        THREE_PER = (THREE_MADE / THREE_ATTEMPTED) * 100
        round(THREE_PER, 1)
    elif THREE_ATTEMPTED == 0:
        THREE_PER = 0
```

```
    FT_MADE = input("Free throws made> ")
    if FT_MADE == "":
        FT_MADE = 0
    elif FT_MADE.isnumeric() is False:
        FT_MADE = input("Please enter a number> ")
    FT_MADE = int(FT_MADE)
    FT_ATTEMPTED = input("Free throws attempted> ")
    if FT_ATTEMPTED == 0 or "":
        FT_ATTEMPTED = 0
    elif FT_ATTEMPTED.isnumeric() is False:
        FT_ATTEMPTED = input("Please enter a number> ")
    FT_ATTEMPTED = int(FT_ATTEMPTED)
    if FT_ATTEMPTED > 0:
        FT_PER = (FT_MADE / FT_ATTEMPTED) * 100
        round(FT_PER, 1)
    elif FT_ATTEMPTED == 0:
        FT_PER = 0
    FOULS = input("Fouls> ")
    if FOULS == "":
        FOULS = 0
    elif FOULS.isnumeric() is False:
        FOULS = input("Please enter a number> ")
    int(FOULS)

    NEW_GAME = [DATE, OPPONENT, MINUTES, POINTS, REBOUNDS, ASSISTS, STEALS, BLOCKS, TURNOVERS, FG_MADE, FG_ATTEMPTED,
                FG_PER, THREE_MADE, THREE_ATTEMPTED, THREE_PER, FT_MADE, FT_ATTEMPTED, FT_PER, FOULS]

    for i in range(len(NEW_GAME)):
        if NEW_GAME[i] == '""':
            NEW_GAME[i] = 0
            for j in range(len(NEW_GAME)):
                if NEW_GAME[i][j].isnumeric():
                    NEW_GAME[i][j] = int(NEW_GAME[i][j])
```

```
    if ROW_INDEX == "" or 0:
        ROW_INDEX = input("Please choose a valid option> ")
    elif ROW_INDEX.isnumeric() is False:
        ROW_INDEX = input("Please choose a valid option> ")
    elif int(ROW_INDEX) > len(ALLPLAYERS):
        ROW_INDEX = input("Please choose a valid option> ")
```

The program has some forms of error checking. If the user enters a blank string into the add row option, the program will input 0 into the database. If the user enters a non-number character, the program will redirect the user and warn them to enter a number.

For loops:

When the program adds the inputted information, a for loop is used to add all of the individual indexes in the list.



```
def getPlayerID():
    """
    User enters the id to search for a player
    :return: str
    """

    global CURSOR

    PLAYERS = CURSOR.execute("""
        SELECT
            name
        FROM
            sqlite_schema
        WHERE
            type = 'table'
    ;""").fetchall()

    ALLPLAYERS = list(sum(PLAYERS[1:], ()))

    for i in range(len(ALLPLAYERS)):
        print(f"{i + 1}: {ALLPLAYERS[i]}")

    ROW_INDEX = input("> ")
```

The program also uses a for loop to get the id of the row for the user to select a row within a table.

Word count: 597